

Attribute Based Encryption



What is Attribute-Based Encryption?

❌ Old Way (Traditional)

You encrypt a file FOR a specific person.

Example: Encrypt FOR Alice

Problem: What if 100 people need access?

✅ New Way (ABE)

You encrypt for a DESCRIPTION of a person.

Example: Encrypt for anyone who is
{CSE + PDEU + 2021 Batch}

All matching students can access it!



Key Idea: Data is locked to a DESCRIPTION of a user, not the user's identity.

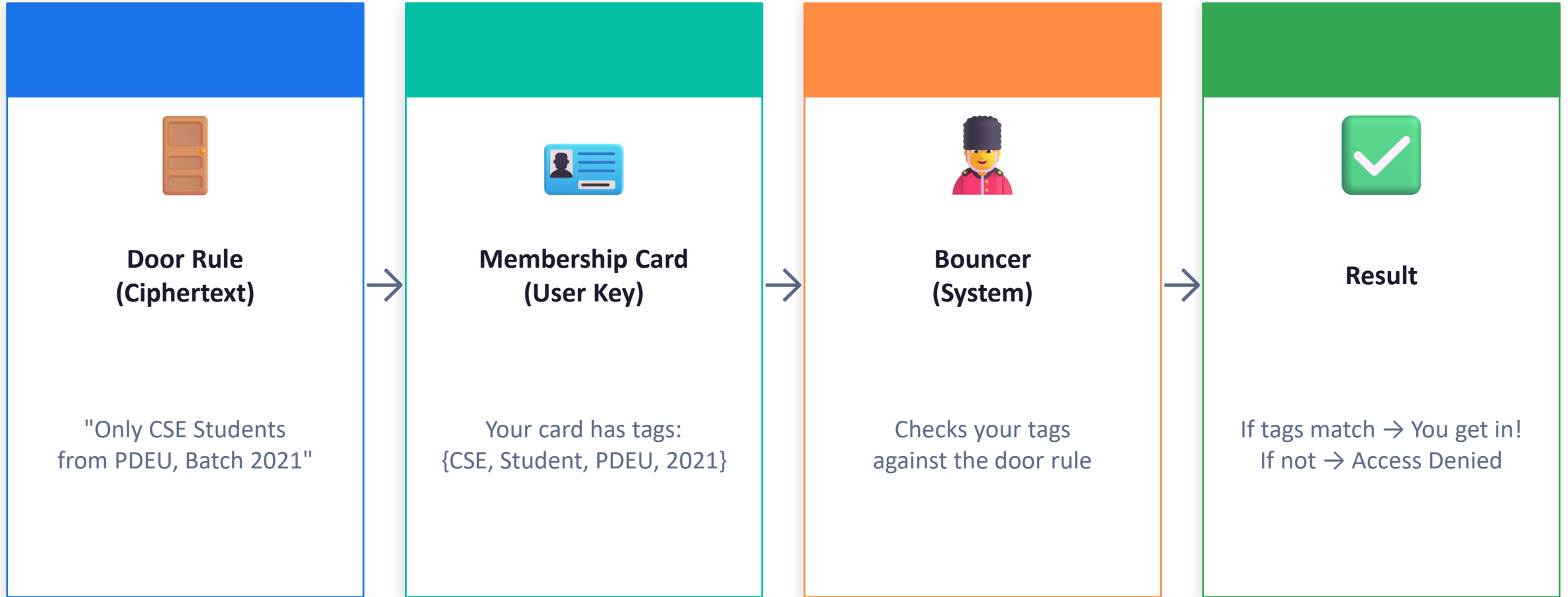
Attribute Based Encryption

- Solves the problem of encrypting data for multiple users at once
- Instead of names → uses attributes.
- In such a system, The decryption of a ciphertext is possible only if the set of attributes of the user key matches the attributes of the ciphertext.
- An access tree (A structure that defines rules using AND / OR) is generated which describes the attributes of a user required to decrypt a ciphertext.
- For example : A message may be encrypted for users who are “CSE” “student” of “PDEU” enrolled in “2021”

User must possess attributes : {CSE, student, PDEU, 2021} to decrypt the target message.



Analogy: The Exclusive Club



Example: Alice {CSE, PDEU, 2020} ❌ | Bob {CSE, PDEU, 2021} ✅

Attribute Based Encryption (continue)

- Two variations: The primary difference lies in where the Access Policy (the rules for decryption) is stored :
 - Ciphertext-policy attribute based encryption
 - Key-policy attribute based encryption
- In CP-ABE user's key depends upon the attributes he possess, and **message is encrypted using an access policy**
- In KP-ABE user's **key is derived from the access policy** required to decrypt a message, and encryption of message depends upon the attributes required to decrypt the message.

CP-ABE Example: University Grading Imagine a professor at PDEU wants to share a grade sheet.

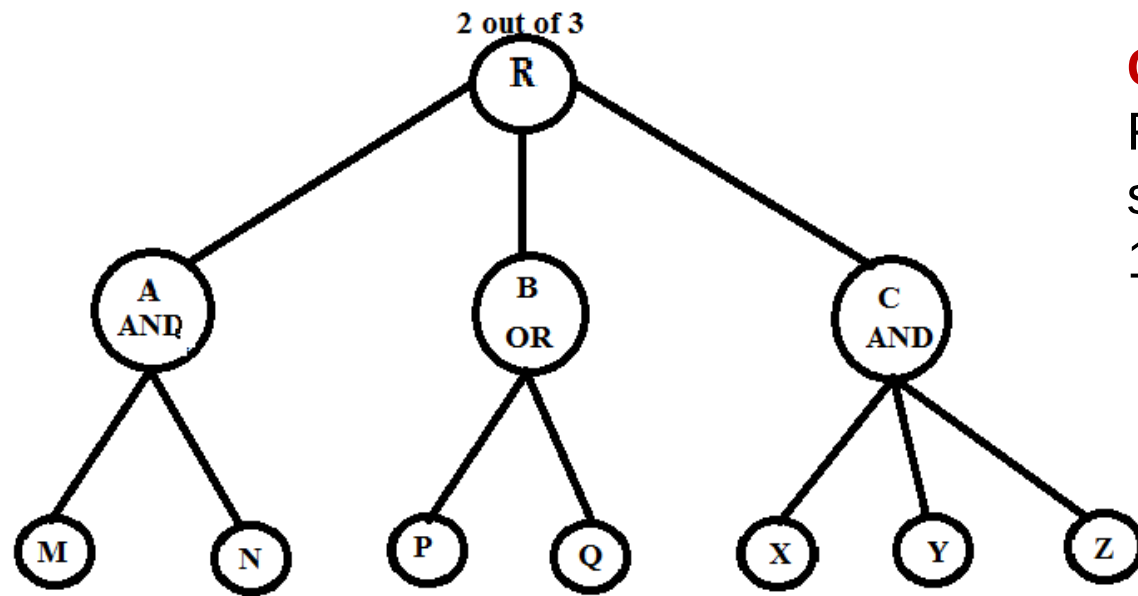
- **The Policy (on the file):** "Department: CSE" **AND** "Batch: 2021".
- **The User Key:** Alice has a key with attributes {CSE, Student, PDEU, 2021}.
- **Result:** Alice can decrypt because her attributes satisfy the file's policy.

KP-ABE Example: Intelligence Agency Imagine a confidential government database where documents are constantly added.

- **The Data:** A file is uploaded and simply tagged with attributes like {TOP SECRET, NUCLEAR, ASIA}.
- **The User's Key (The Policy):** An agent is given a key with a built-in rule: "**TOP SECRET**" **AND** ("**ASIA**" **OR** "**EUROPE**").
- **Result:** The agent can open any document tagged with these attributes because their key has the specific "routing" authority to do so.

Access Policy Tree

- Attributes are leaves.
- Each non-leaf node represents a threshold gate like AND, OR, k out of m .
- Root node has the secret value that is split and its portions are assigned to different attributes based on policy.
- Secret value assigned to root node is used to encrypt the message. The portions of secret value assigned to each leaf node are included either as ciphertext components or user's key components depending upon the scheme (CP-ABE/KP-ABE).



Construction of Access tree

For each non-leaf node 'x' a threshold value ' k_x ' is specified.

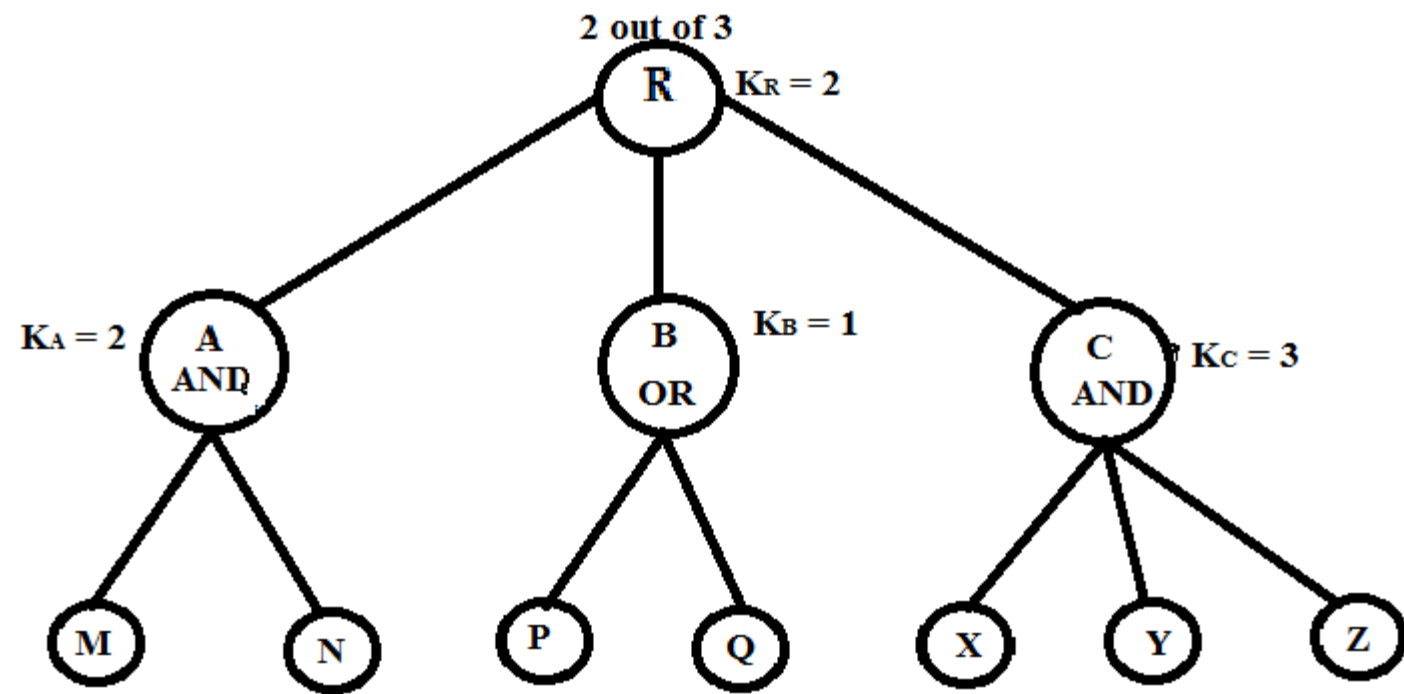
$1 \leq k_x = num$ (num is the no. of children of x)

For AND node $K_x = num$.

For OR node $K_x = 1$

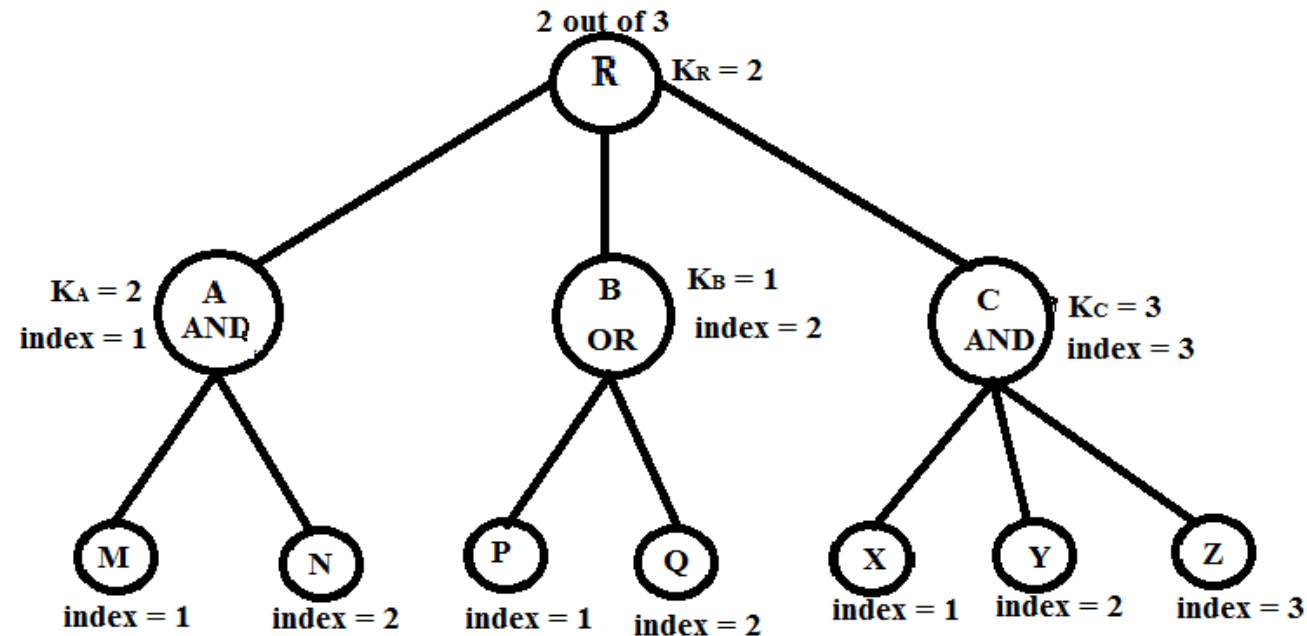
For k out of m node $K_x = k$.

- **Root Node (R): "2 out of 3" Threshold Gate** To satisfy the entire policy, you must successfully satisfy at least **two** of the three branches (A, B, or C).
- **Branch A: "AND" Gate** To satisfy Node A, a user must have **both** attributes **M AND N**.
- **Branch B: "OR" Gate** To satisfy Node B, a user needs either attribute **P OR Q** (or both).
- **Branch C: "AND" Gate** To satisfy Node C, a user must have all three attributes: **X AND Y AND Z**.



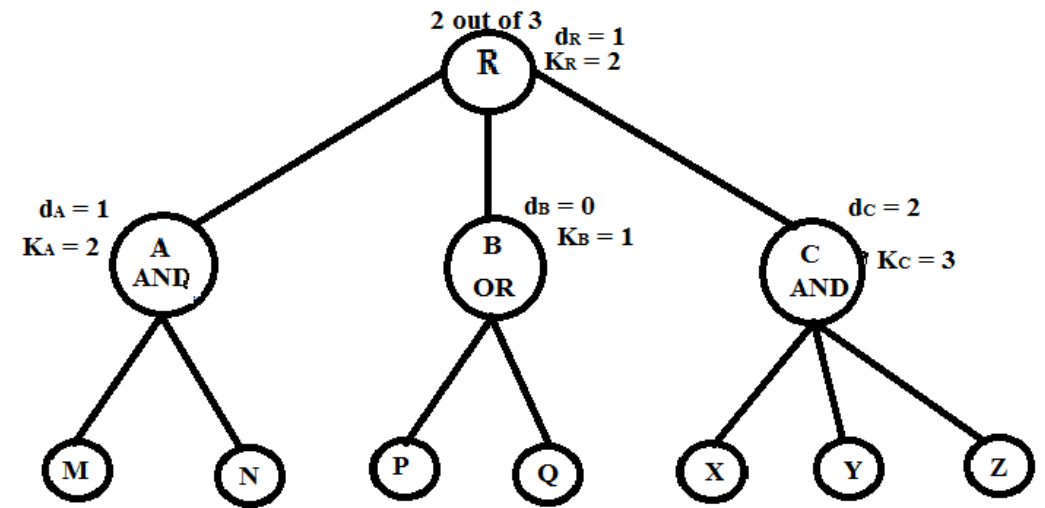
Construction of Access tree (Continue)

- For each parent node its children are assigned index number from 1 to *num*. (*num* is the number of children of a node)



Construction of Access tree (Continue)

- Next step is to generate polynomial for each node.
- Assign a degree for polynomial to each node.
- **For a node x the degree of polynomial is $d_x = k_x - 1$.**



Construction of Access tree (Continue)

- At each node you require $d_x + 1$ number of points to generate a polynomial with degree d_x .
- For root node choose one random value s which is master secret.
- For root node R one point is $(0,s)$. Choose other d_R points. (x_1, y_1) $(x_2, y_2) \dots (x_{d_R}, y_{d_R})$. Total points = $d_R + 1$ where d_R = degree of R
- For each interleaved node one point is $(0, q_{\text{parent}}(\text{index}(x)))$. Choose other d_x points randomly as in previous step.

Construction of Access tree (Continue)

- Generate polynomial for each interleave node using Lagrange Polynomial equation. (For degree n-1)

$$\frac{(x - x_2)(x - x_3) \dots (x - x_n)}{(x_1 - x_2)(x_1 - x_3) \dots (x_1 - x_n)} * y_1 +$$
$$\frac{(x - x_1)(x - x_3) \dots (x - x_n)}{(x_2 - x_1)(x_2 - x_3) \dots (x_2 - x_n)} * y_2 + \dots$$
$$\frac{(x - x_1)(x - x_2) \dots (x - x_{n-1})}{(x_n - x_1)(x_n - x_2) \dots (x_n - x_{n-1})} * y_n$$

Example

Used as a key for encryption

- For Root Node the points are **(0,3)** ; (1,5)

$$q_R = 2x + 3$$

The "Secret" for the Child: The y value at $x = 0$ for each child must match the parent's polynomial value at that child's index.

- Node A:** Its secret is $q_R(1) = 2(1) + 3 = 5$

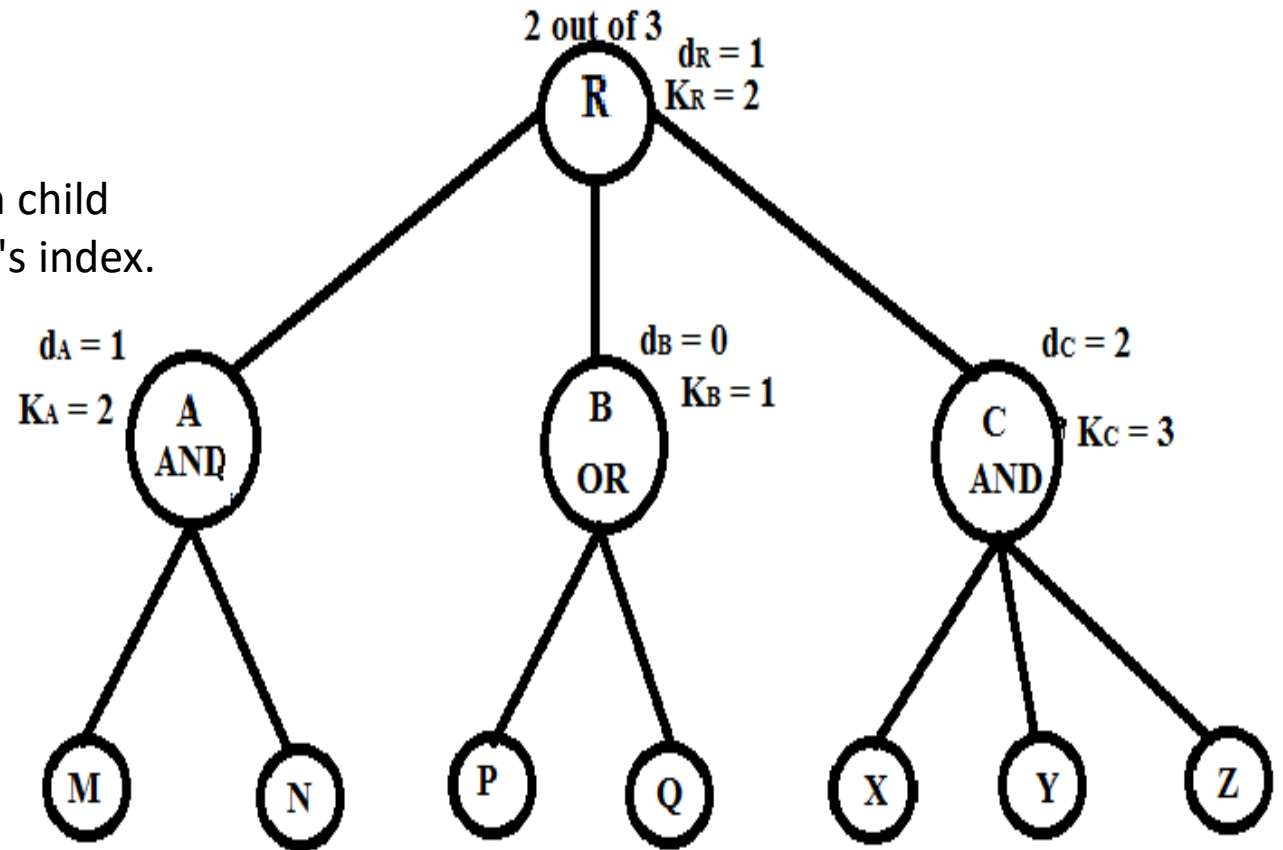
So, Node A's first point is **(0, 5)**.

- Node B:** Its secret is $q_R(2) = 2(2) + 3 = 7$

So, Node B's first point is **(0, 7)**

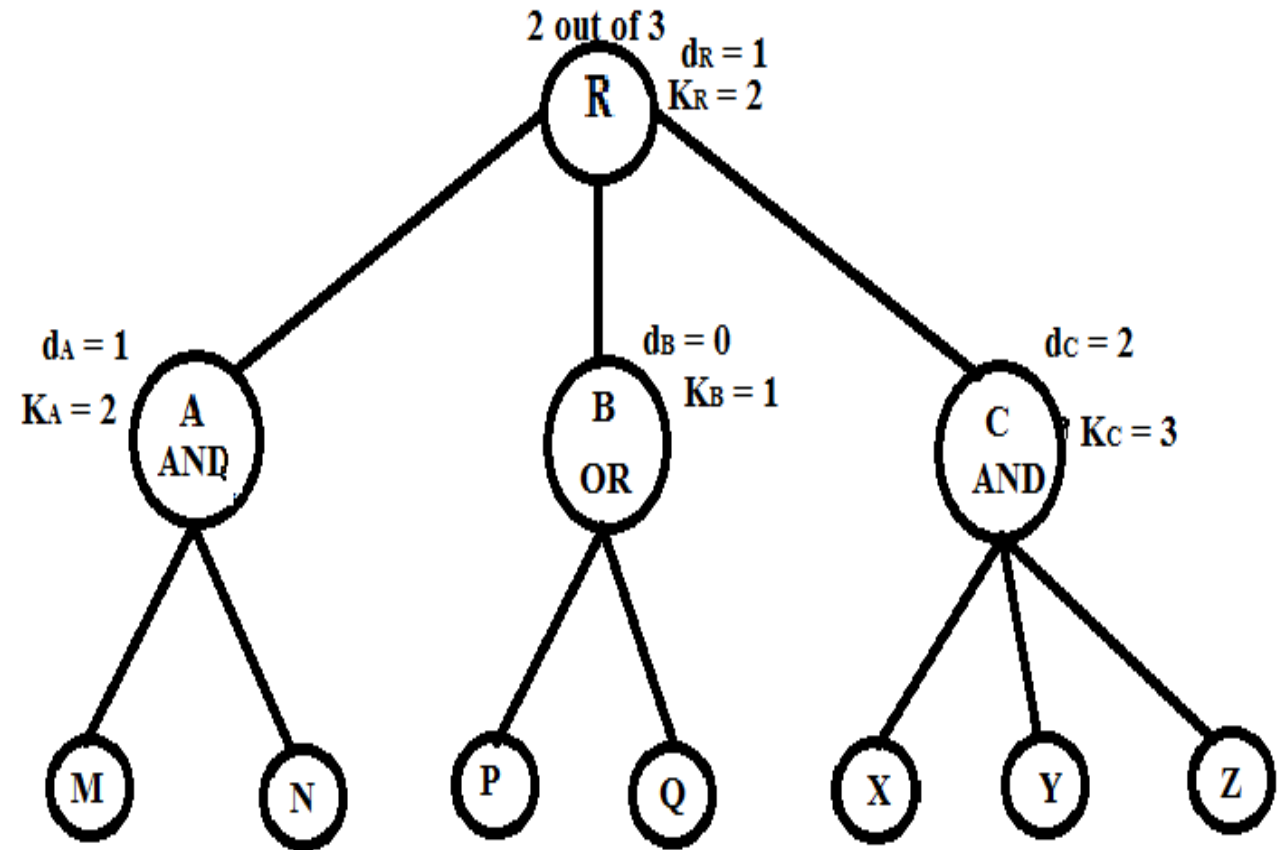
- Node C:** Its secret is $q_R(3) = 2(3) + 3 = 9$

So, Node C's first point is **(0, 9)**



...Example

- For Node A **(0,5)** ; (2,1)
 $q_A = 5 - 2x$
- For Node B **(0,7)**
 $q_B = 7$
- For Node C **(0,9)**; (3,6); (4,1)
 $q_C = -x^2 + 2x + 9$



...Example

- For Node M (0,3)
- For Node N (0,1)
- For Node P (0,7)
- For Node Q (0,7)
- For Node X (0,10)
- For Node Y (0,9)
- For Node Z (0,6)

$$q_M = 3 \quad [q_A(1) = 5 - 2(1) = 3]$$

$$q_N = 1 \quad [q_A(2) = 5 - 2(2) = 1]$$

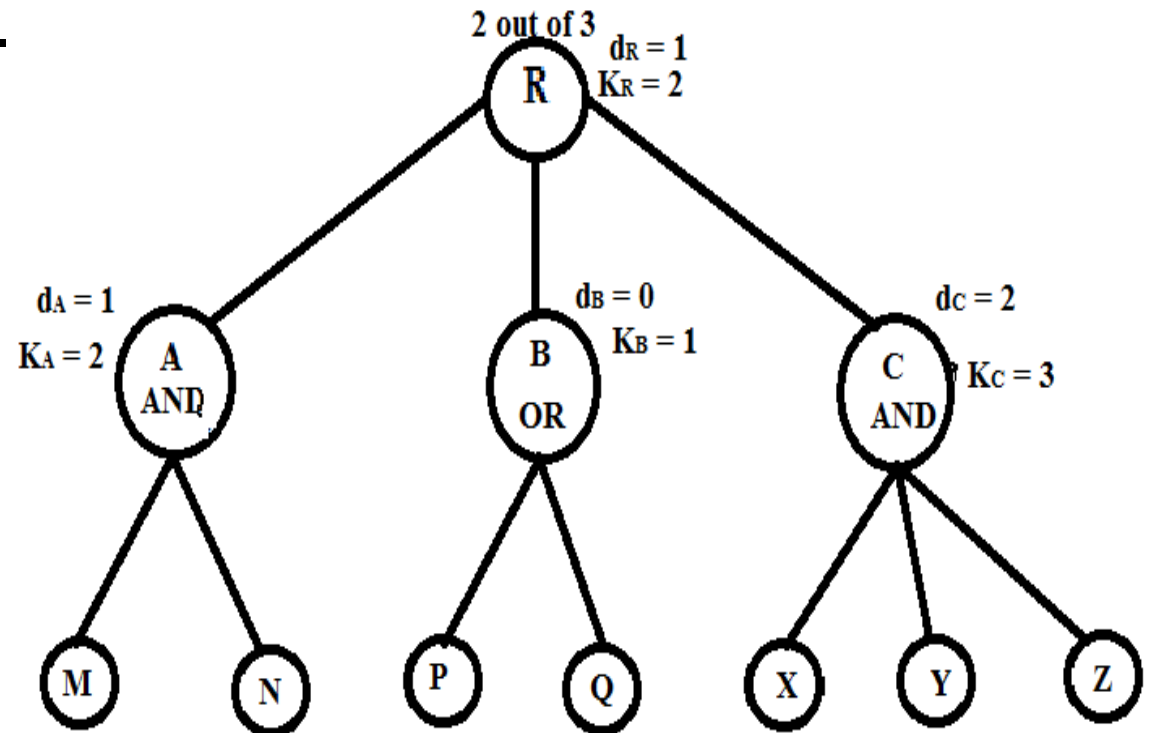
$$q_P = 7 \quad \dots$$

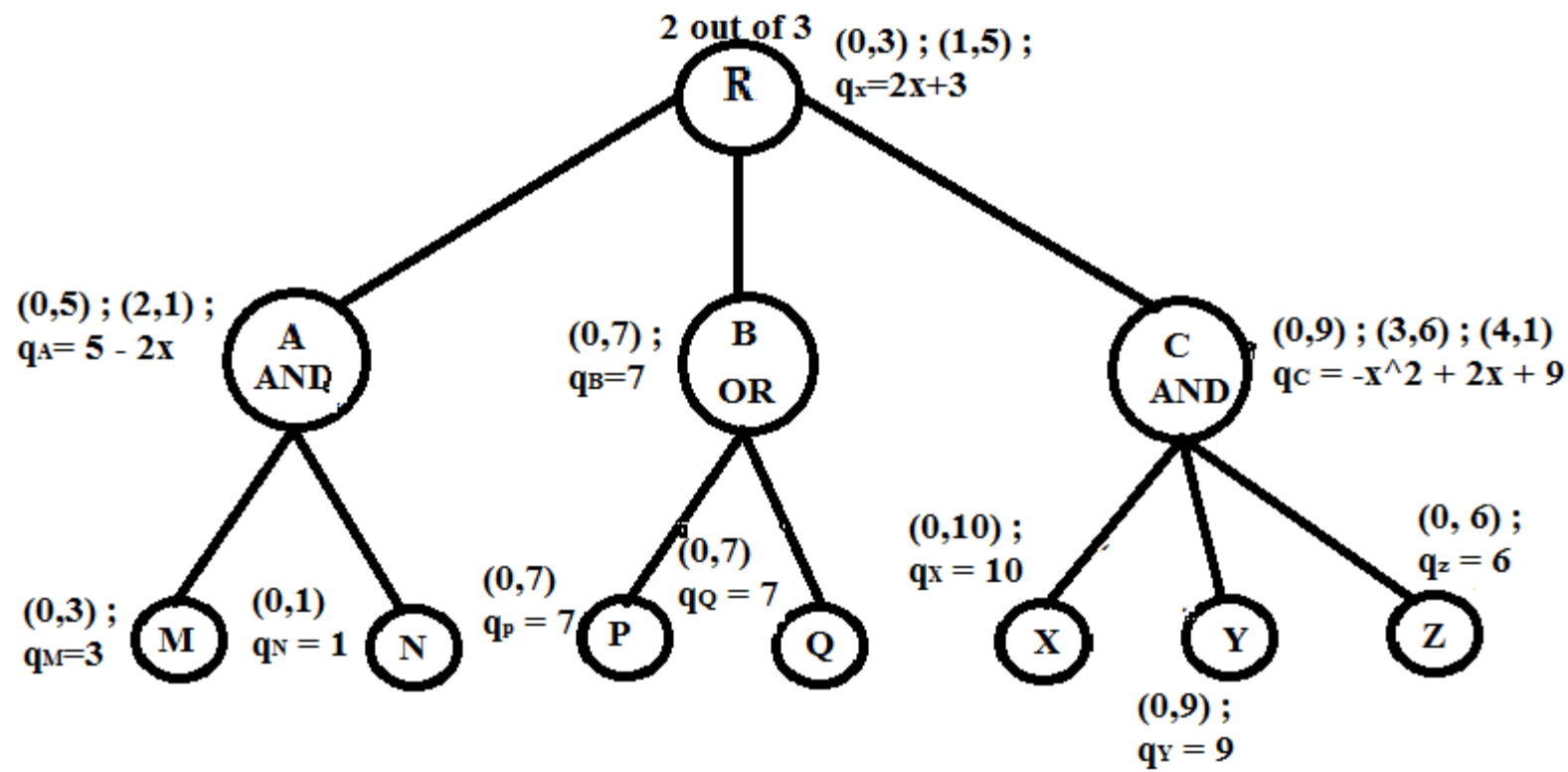
$$q_Q = 7$$

$$q_X = 10$$

$$q_Y = 9$$

$$q_Z = 6$$





Functions Defined

$\text{index}(x)$: It returns index number of a node x .

$\text{parent}(x)$: It returns parent node of x .

$\text{attr}(y)$: It returns the attribute associated with leaf node y .

Bilinear Maps

- G_0 and G_1 : Two multiplicative cyclic groups of prime order p .
- g is a generator of G_0 .
- Then e is bilinear map as defined below :
 $e : G_0 \times G_0 \rightarrow G_1$ such that it satisfies following :
 - For all $u, v \in G_0$ and $a, b \in \mathbb{Z}_p$, we have $e(u^a, v^b) = e(u, v)^{ab}$.
 - $e(g, g) \neq 1$

Cipher Text Policy Attribute Based Encryption

- Setup
- Encryption
- Key Generation
- Key Delegation
- Decryption

CP- ABE

- Setup :

- Input : A security Parameter k .

- Bilinear group G_0 of prime order p with generator g , Two Random exponents α and β from Z_p .

- Output :

- Public Key : $PK = G_0, g, h = g^\beta, f = g^{1/\beta}, e(g, g)^\alpha$

- Master Key : (β, g^α)

CP- ABE Encryption

- Input : Public Key PK, Message M, Access Tree A
- Output : Ciphertext CT

$$\begin{aligned} \text{CT} = \{ & T (\text{Access Policy}), C' = Me(g,g)^{\alpha s}, C = h^s, \\ & \text{For every } y \in Y \text{ (} Y \text{ is the set of leaf} \\ & \text{nodes)} : \\ & C_y = g^{q_y(0)}, C'_y = H(\text{att}(y))^{q_y(0)} \\ & \} \end{aligned}$$

CP-ABE

- Key Generation:

- Input : Master Key MK, Set of Attributes S
- Output : private Key SK

$$SK = (D = g^{(\alpha + r)/\beta},$$

For every attribute $j \in S$ (S is the set of all attributes) $D_j = g^{r \cdot H(j)}$, $D'_j = g^{r \cdot j}$
)

CP-ABE

- Delegate:

- Input : Secret Key SK, Subset of Attributes S'
- Output : private Key SK'

$$SK = (D^{\wedge} = Df^{r'},$$

For every attribute $k \in S'$,

$$D^{\wedge}_k = D_k \cdot g^{r'} \cdot H(j)^{rk'}, D^{\wedge}_k = D'_k \cdot g^{rk}$$

)

Lagrange Coefficient Used In decryption

- $\Delta_{i,s}(x) = \prod_{j \in S, j \neq i} ((x-j)/(i-j))$

CP- ABE Decrypt

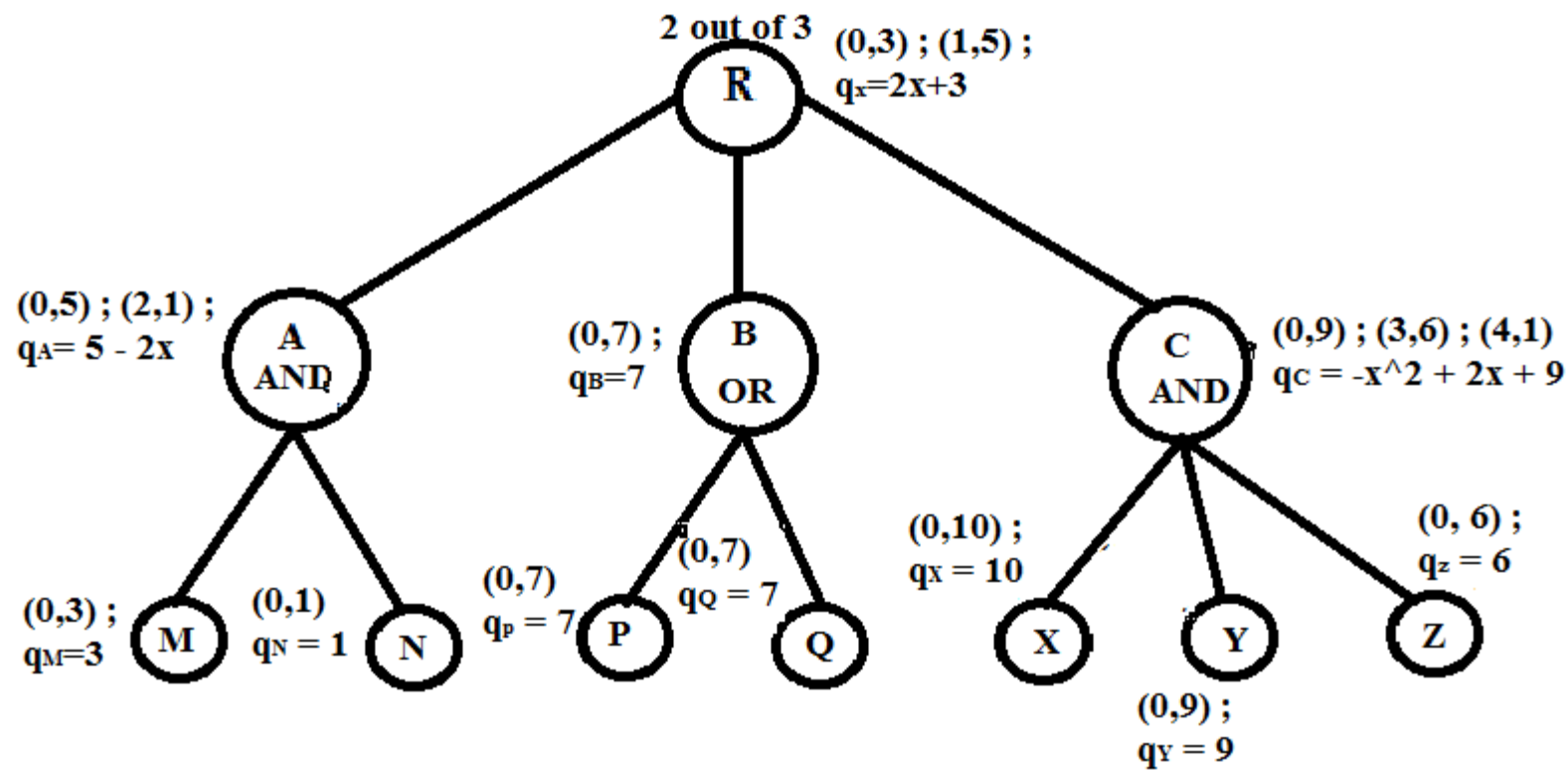
- Input : Cipher text CT, Secret Key SK
- Procedure:
 - DecryptNode Procedure for every leaf node z (which the user possess) $i = \text{att}(z)$

$$\frac{e(D_i, C_z)}{e(D'_i, C'_z)}$$

$$= e(g, g)^{r_{qz}(0)} \quad \text{call this result as } F_z$$
 - For non leaf nodes x if z denotes its child and S_x is the set of its children then

$$F_x = \prod_{z \in S_x} F_z^{\Delta_{i, s'_x}(0)} \quad S'_x \text{ is the set of indexes of nodes which are in } S_x. i = \text{index of } z$$

$$F_x = e(g, g)^{r_{qx}(0)}$$
 - Recursive procedure of above step will return $A = e(g, g)^{rs}$



CP- ABE Decrypt Example

- Let's say the user is having keys for M,N, and P attributes
- DecryptNode Procedure for M,N and P will give following values
 - $F_M = e(g,g)^{3r}$
 - $F_N = e(g,g)^r$
 - $F_P = e(g,g)^{7r}$

CP- ABE Decrypt Example Continue

- $F_M = e(g,g)^{3r}$
- $F_N = e(g,g)^r$
- $F_P = e(g,g)^{7r}$

- For non leaf nodes A Following calculation will be held

$$\begin{aligned} S_A &= \{M, N\} \\ F_A &= \prod_{z \in S_A} F_z^{\Delta_{i,s'A}(0)} \quad S'_A = \{1, 2\} \\ &= (F_M^{\Delta_{1,s'A}(0)}) (F_N^{\Delta_{2,s'A}(0)}) \\ \Delta_{1,s'A}(0) &= (0-2)/(1-2) = 2 \\ \Delta_{2,s'A}(0) &= (0-1)/(2-1) = -1 \\ F_A &= (F_M^2)(F_N^{-1}) = e(g,g)^{6r} e(g,g)^{-r} \\ &= e(g,g)^{5r} \\ &= e(g,g)^{rqA(0)} \end{aligned}$$

CP- ABE Decrypt Example Continue

- $F_M = e(g,g)^{3r}$
- $F_N = e(g,g)^r$
- $F_P = e(g,g)^{7r}$

- For non leaf nodes B

$$F_B = F_P \\ = e(g,g)^{7r}$$

$$S_B = \{P\} \\ S'_B = \{1\}$$

- For root node R Following calculation will be held

S_R

$$A = F_R = \prod_{z \in S_R} F_z^{\Delta_{i,s'_R}(0)} \quad S'_R = \{1,2\}$$

$$= (F_A^{\Delta_{1,s'_R}(0)}) (F_B^{\Delta_{2,s'_R}(0)})$$

$$\Delta_{1,s'_R}(0) = (0-2)/(1-2) = 2$$

$$\Delta_{2,s'_R}(0) = (0-1)/(2-1) = -1$$

$$F_A = (F_A^2)(F_B^{-1}) = e(g,g)^{10r} e(g,g)^{-7r}$$

$$= e(g,g)^{3r}$$

$$= e(g,g)^{rs}$$

CP- ABE Decrypt

- $C' / (e(C, D) / A)$
 $= C' / (e(h^s, g^{(\alpha + r)/\beta}) / e(g, g)^{rs})$
 $= M$

References

- John Bethencourt, Amit Sahai, and Brent Waters. "Ciphertext-policy attribute-based encryption." In *2007 IEEE symposium on security and privacy (SP'07)*, pp. 321-334. IEEE, 2007.
- Amit Sahai, and Brent Waters. "Fuzzy identity-based encryption." In *Advances in Cryptology-EUROCRYPT 2005: 24th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Aarhus, Denmark, May 22-26, 2005. Proceedings 24*, pp. 457-473. Springer Berlin Heidelberg, 2005.